

ggRobot 功能与技术盘点

项目：ggRobot — 灵犀 X2 人形机器人 Web 控制台 v2.0

整理日期：2026-07-28

依据：逐文件核对真实源代码（avoidance.py、node.py、task/、ws/stream.py、**main.py**、retry.py、前端 x2model 等）。

标记约定：

- 【现状】：代码里真实存在的实现（含半成品、接口空挂会如实说明）。
- 【可延伸】：基于现有基础可往下做成完整方案的具体方向，供商议参考。

一、项目定位与整体架构

ggRobot 是运行在 X2 人形机器人 Orin 计算单元（PC2）上的 Web 控制台，用户通过浏览器操控机器人的语音、动作、遥控、表情、媒体等功能，并实时查看传感器状态与 3D 孪生模型。

硬件拓扑（三计算单元）：

单元	IP	角色
PC1 运控单元	10.0.1.40	运动控制大脑（站立行走），不可改动
PC2 Orin NX	10.0.1.41	本项目运行位置
PC3 交互单元	10.0.1.42	屏幕与扬声器，放音视频文件

软件栈：后端 FastAPI + rclpy/ROS2（Python），前端 Vue3 + TypeScript + Three.js + Naive UI。

核心通信架构：

- 命令队列模式：浏览器 → FastAPI(unicorn 线程) → `queue.Queue` → rclpy 线程消费 → ROS2 Service 调用，结果通过 `Future` 回传。
- 双通道推送：REST（按需指令）+ WebSocket（200ms 传感器 / 100ms 相机帧 / 键盘遥控）。
- 双线程：MultiThreadedExecutor 独立 spin 线程 + 独立 cmd 命令线程（5ms 轮询命令队列）。

二、功能面（当前能力清单）

标注：✅ 可用 | 🟡 接口已通但浅 / 依赖外部 | ❌ 缺口

运动控制

- ✅ 速度遥控：浏览器键盘 WASD/QE，50Hz 持续发送三向速度（前/横/转），松手即停。
- ✅ 预设动作：按身体区域（左臂/右臂/双臂/全身）触发预设动作 ID。
- ✅ 灵创动作资源：查询并执行机器人自带动作资源包（onnx 全身动作 / 手臂动作）。
- ✅ 运动模式切换：PASSIVE / DAMPING / JOINT / STAND / LOCOMOTION 等模式字符串切换。

语音与表情

- ✅ TTS 语音播报：文字转语音，带优先级与可中断标志。
- ✅ 面部表情：21 种表情分 4 类（基础/情绪/卖萌/其他），支持单次/循环。
- ✅ 音量 / 静音：查询与设置。

媒体播放

- ✅ 音频播放：播放 PC3 上的音频文件（自动路由到 PlayAudioFile）。
- ✅ 视频播放：按扩展名自动路由到 PlayVideo。
- ✅ 脸屏 Logo：循环播放 logo 视频到脸部屏。

感知查看

- ✅ 多路相机：RGBD 前视 / RGB 后视 / 双目左 / 双目右，实时切换查看。
- ✅ 传感器状态：电池（电量/电压/电流/温度/功率）、关节（位置/速度）、IMU（三轴加速度）。
- ✅ 3D 数字孪生：浏览器内渲染机器人模型，关节实时驱动、部件点击高亮。
- ✅ 系统状态：运动控制状态机、运动模式、连接拓扑。

自动化

- ✅ 任务编排：低代码 workflow，按步骤序列执行（TTS/动作/表情/速度/模式/音量/媒体/相机/灵创/HTTP 调用/等待），支持变量传递与中断。
- ✅ 激光避障：独立 ROS2 节点，filter / auto / dry_run 三种模式。

语音采集

- **✗ mic / VAD 采集**：前端 `Voice.vue` 有完整 UI (VAD 四态可视化、PCM 段下载)，但后端 `routes/mic.py` 实际是运动模式路由（遗留 bug），采集接口缺失。

三、技术面（实现细节与可延伸方向）

3.1 分布式调度架构（多线程 + 命令队列 + Future 桥接）

【现状】

- 主线程启动 rclpy 线程：`MultiThreadedExecutor.spin()` 持续分派传感器 / service / 定时器回调。
- 独立 cmd 命令线程（daemon, 5ms 轮询）消费命令队列，执行耗时命令（动作序列、任务编排、等待 TTS）。
- FastAPI 的 async handler 通过 `CommandQueue.put()` 拿到 `Future`，阻塞等待结果；rclpy 线程执行后 `future.set_result()` 回传。
- 设计动机（代码注释记录）：耗时命令若在 rclpy 线程同步执行，会长时间不调 `spin`，传感器回调被饿死，表现为数据冻结。

【可延伸】

- 命令优先级与抢占（当前是 FIFO 顺序执行）。
- 命令超时/熔断与回滚（当前只有重试，无事务性回滚）。
- 多机器人/多实例的命令路由（当前单节点）。

3.2 高频速度控制（常驻定时器 + 目标速度原子替换）

【现状】

- `__init__`（rclpy 线程）创建一次 50Hz 定时器（0.02s），永不 **create/destroy**。
- 命令线程的 `_do_velocity()` 只做一件事：`self._vel_target = (forward, lateral, angular)`，元组整体替换（GIL 原子，无锁）。
- 定时器回调 `_publish_velocity()` 读 `_vel_target` 发布；全零目标 = 持续发零 = 停车。
- 输入源懒注册：首次发速度时注册 `web_ui`（priority=40, timeout=1000ms），未注册时不发。
- 设计动机（代码注释记录）：早期每次发速度都 **create/destroy** 定时器，与 `executor.spin()` 竞态导致卡死、传感器停更。

【可延伸】

- 多输入源（手柄 / 语音 / 规划器 / Web）的优先级仲裁与平滑混权。
 - 速度软启动/软停止曲线（当前是阶跃）。
 - 与激光避障节点（已具备 filter 模式）串联形成安全速度链。
-

3.3 跨板 Service 可靠性（重试模板）

【现状】

- `retry.call_with_retry`：默认 8 次重试、每次 0.25s 超时。
- 关键设计：只做 `future.done()` 轮询，不重入 `rclpy.spin_until_future_complete`（代码注释记录：那会与 `MultiThreadedExecutor` 抢锁死锁）。
- 所有跨计算单元的 Service 调用统一走该模板。

【可延伸】

- 自适应重试（按 Service 类型 / 历史成功率动态调参）。
 - 跨板链路质量监测与告警。
-

3.4 激光避障安全过滤器（点云处理链 + 状态机）

说明：RANSAC 地面拟合、体素降采样、扇区分区均为成熟通用技术（PCL/Open3D 常见），下面如实陈述项目组合使用的方式。

【现状】

- 处理链：PointCloud2 解析 → ROI 裁剪（机器人坐标系 6 维边界）→ 体素降采样（0.05m，压到约 500 点）→ 地面去除（RANSAC 法向量 $Z > 0.3$ 拟合 + 高度过滤，取并集）→ 9 扇区分区（前方 120° FOV）→ EMA 平滑。
- 决策状态机：IDLE / GO / SLOW / AVOID / TURN / STOP / TRAPPED，输出安全速度。
- 工程细节：方向锁死（左右差距 $< 0.15\text{m}$ 时沿用上次方向，防左右打架）；原地转向阈值（`turn_in_place_distance`）；两侧都堵死进入 TRAPPED 停住。
- 三种模式：filter（过滤 `/cmd_vel`）、auto（独立前进 + 避障）、dry_run（空跑不发包）。
- 输入源注册/注销流程（action 1001 注册 / 1003 注销）。
- 回调防堆积：`_busy` 标志，还在处理上一帧就跳过新帧。

【可延伸】

- 3D 高度分层避障（当前只用前方水平扇区）。
 - 动态障碍检测与速度估计。
 - 视觉/深度图融合避障。
 - 与 SLAM 建图结合做自主导航。
-

3.5 多源传感器聚合 + 执行器健康度监测

【现状】

- 三个 ROS2 订阅（PmuState / JointStateArray / Imu）各自把消息规整成扁平 dict 缓存（电池拆 5 字段、关节映射 name/position/velocity、IMU 取三轴加速度）。
- `sensor_pusher` 每 200ms 把三者合并成一个 JSON 推送（降频到 5Hz，降低 WS 流量）。
- 健康度探针：每约 10s 打印传感器回调计数 b/a/i；计数不增长即判定 `executor.spin` 卡死（代码注释记录：用于定位"前端值不变"是回调饿死还是网络问题）。
- 首条消息订阅确认（每个回调首条打日志）。

【可延伸】

- 传感器数据落盘导出（形成数据集，对接训练/数据分析）。
 - 多模态时序对齐（电池/关节/IMU/相机统一时间轴）。
 - 异常检测与告警（电流突变、IMU 跌倒预判等）。
-

3.6 实时数据推送通道（双频 + 时间戳帧 + 中断检测）

【现状】

- 双频后台任务：相机帧 100ms、传感器 200ms，仅在 WS 客户端时推送。
- 相机帧格式：4 字节大端时间戳（ms）+ JPEG 二进制；前端切掉前 4 字节解析，上一帧 `revokeObjectURL` 释放内存。
- 帧中断检测：首帧不告警（避免冷启动误报），曾有帧后中断超 5s 才告警一次（`_warned` 防重复）。
- 死连接清理：广播失败即从客户端集合移除。

【可延伸】

- 自适应码率/帧率（按网络质量动态调整）。
 - 推送链路本身的 QoS 保障与拥塞控制。
-

3.7 多路相机异构 QoS 与跨板发现容忍

【现状】

- 声明式相机配置表 (id → label / topic / qos), 覆盖 4 路。
- 实际订阅使用内置 `qos_profile_sensor_data` (代码注释记录: 手搓 QoSProfile 字段不全导致 DDS 协商失败、Subscription count=0)。
- 不预检 topic 是否已被 DDS 发现, 直接 `create_subscription`, 等 publisher 自然匹配。
- `process_commands` 中相机兜底: 60s 仍未收到帧只告警不主动切换 (注释记录: DDS 没发现 publisher 时切换也没用)。

【可延伸】

- 多路相机同屏拼接/画中画。
 - 相机帧与关节状态融合的远程临场感。
 - 自动选择最优可用相机源。
-

3.8 任务编排引擎 (低代码 workflow)

【现状】

- 步骤类型: `tts / motion / emoji / velocity / wait / mode / volume / media / camera / linkcraft / http`, 注册表驱动。
- 变量系统: `{{var.path}}` 模板渲染 (递归处理 str/dict/list, 支持 dict 取键 + list 取下标); HTTP 响应的 `data` 字段自动提升到顶层; `save_as` 把响应存入上下文供后续步骤引用。
- 流程控制: `expect_code` 业务码校验失败抛 `StepAbort` 中止整个任务 (区别于普通异常); `threading.Event` 实现任务级软中断; 任务结束兜底发全零速度。
- 能力自描述: `CAPABILITIES` 表 (label/icon/params/hint) 供前端数据驱动生成配置表单。
- TTS 步骤支持并行挂载多个动作与表情。

【可延伸】

- 条件分支 / 循环 / 并行步骤 (当前仅顺序)。
 - 任务录制与回放 (记录人工操作序列 → 生成可复用任务)。
 - 参数化模板库与任务版本管理。
 - 与传感器数据落盘结合, 作为示教/数据采集的编排底座。
-

3.9 语音-动作同步（双模态完成判定）

【现状】

- TTS 完成判定 (`wait_tts_done`)：主轨用 `estimated_duration` 估时 + 0.3s 余量；副轨订阅 `PlayStateChange`，期间收到 `STOPED` 提前返回；超时 20s 降级。
- 动作完成判定 (`wait_motion_done`)：轮询 `GetMcAction.status`，用"先进入 `RUNNING` 再退出 `RUNNING`"判定，避免初始 `IDLE` 误判完成。
- TTS 步骤 `motion_wait` 开关：开 → 取 `max(TTS, 动作)` 等都完成；关 → 语音播完即 `stop_motion()`（切回 `STAND` 打断动作）。

【可延伸】

- 多语种 / 情感语音与表情的联动编排。
 - 语音打断队列与优先级调度。
 - 语音-口型-动作的三模态对齐。
-

3.10 3D 数字孪生运动链（URDF → JSON + 双层 Group）

【现状】

- 离线把 URDF 转成精简 JSON（只留 `name / mesh / origin / axis / limit / joint_type`）。
- 运行时递归构建运动链：每个 link 一个位置 Group；若是 revolute 关节，在位置 Group 内再套一个旋转层 Group，子树挂到旋转层。
- 关节驱动： `setJointAngle` 只需改旋转层 Group 的一个 quaternion (`setFromAxisAngle`) 即驱动整个子链。
- 坐标系换轴（ROS 右手系 → Three.js）、STL 并行加载 + `mergeVertices` 法线重算、材质 clone 防高亮串扰、link→joint 反查表驱动点击高亮。

【可延伸】

- 录制的关节轨迹在孪生体上回放（对接数据采集/训练）。
 - 物理仿真与碰撞预演。
 - 多机器人型号的通用 URDF→JSON 流水线。
-

3.11 跨板音视频资源分发

【现状】

- 媒体上传到 PC2 暂存 → `sshpas`/`scp` 同步到 PC3 `/agibot/data/home/agi/media/` → 自动 `chmod` (目录 755 / 文件 644) → 返回码校验。
- 免密/密码自适应 (`_pc3_prefix` 按是否配密码决定用 `sshpas` 还是依赖免密)。
- Logo 首次 `show` 时懒同步; `PlayVideo` 无 `stop` 字段, 用切回待机表情覆盖式关闭。

【可延伸】

- 资源版本管理与增量同步。
- 跨板资源一致性校验。

3.12 WebSocket 半开连接看门狗（前端）

【现状】

- 心跳: 5s 发 `{type:'ping'}`, 服务端回 `{type:'pong'}`。
- 看门狗: 15s (约 3 次心跳) 内未收到任何消息 (含 `sensor/camera/pong`) 即判定半开连接, 主动 `ws.close()` 触发重连 (3s 延迟)。
- 设计动机 (注释记录): 解决"socket 还在但数据不来, 表现为传感器数据冻结"的真实场景。

【可延伸】

- 断线期间指令的本地缓存与重连后补发。
- 多链路冗余 (WS + 轮询降级)。

四、研发资料输出能力（可供商议/交底使用的事实素材）

项目能直接提供以下材料：

资料类型	具体内容	出处
详尽技术注释	各技术点有"问题 → 踩坑过程 → 解法"的中文注释	全代码库
架构图/数据流图	命令队列、双通道推送、多线程架构的 ASCII 图	<code>CLAUDE.md</code>
状态机图	避障 7 态迁移图	<code>avoidance.py</code>
配置参数表	ROI / 距离阈值 / 速度限制 / 扇区参数	<code>avoidance.py</code> CFG

资料类型	具体内容	出处
运行时日志	启动日志、避障决策、传感器计数(b/a/i)、相机帧fps、TTS 估时	各 <code>logger.info</code>
接口清单	已接入的 16 个 ROS2 接口 + QoS 配置	<code>CLAUDE.md</code> + <code>node.py</code>
踩坑档案	QoS 协商失败、 <code>executor</code> 卡死、跨板媒体路径、SDK 字段笔误等	代码注释 + 记忆库

五、能力现状汇总

已完整落地（可作为事实基础）：双线程调度架构、高频速度控制、跨板重试、激光避障过滤器、多源传感器聚合与健康度探针、实时推送通道、多路相机切换、任务编排引擎、语音-动作同步、3D 孪生运动链、跨板媒体分发、WebSocket 看门狗。

接口已通但未深做：

- 灵创动作资源（仅查询 + 执行，无版本/编辑管理）。
- 脸屏 Logo（覆盖式关闭，非显式停止接口）。
- 相机（单路切换，无多路同屏/融合）。

缺口（需补开发）：

- **训练侧：**项目只有动作执行侧，**无关节轨迹录制 / 标注 / 数据集导出**。
- **mic 采集后端：**`routes/mic.py` 实为运动模式路由（遗留），VAD 采集接口缺失，前端 UI 已就绪。

上述"接口已通但未深做"与"缺口"两项，是可在会议中讨论是否往下延伸做成完整方案的方向。